

Práctica 1: Programando el Perceptrón de Rosenblatt

Contexto y Objetivos

Hasta ahora hemos estudiado la teoría de la neurona artificial y cómo Frank Rosenblatt diseñó el primer algoritmo de aprendizaje supervisado. Ahora es vuestro turno de "abrir la caja negra". En esta práctica, implementaréis desde cero la clase Perceptron en Python.

Objetivos de aprendizaje:

- Traducir fórmulas matemáticas ($z = w \cdot x + b$) a código numpy.
- Implementar el algoritmo de descenso de gradiente básico (regla de aprendizaje del Perceptrón).
- Comprender el concepto de "época", "tasa de aprendizaje" y "convergencia".

Instrucciones de Implementación

El fichero perceptron_lab.py contiene el esqueleto de la clase. Tu misión es localizar los comentarios marcados como **TODO** y completar el código que falta.

Paso 1: La "Entrada Neta" (TODO 2)

Busca el método `net_input(self, X)`.

Una neurona primero debe "pesar" las entradas. Matemáticamente, esto es la suma ponderada más el sesgo:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

O en notación vectorial:

$$z = W \cdot X + b$$

- **Tu tarea:** Implementa el cálculo de "z".
- **Restricción:** No uses bucles for para multiplicar los pesos por las entradas uno a uno. Debes usar la potencia de **Numpy** para realizar un **producto escalar (dot product)** entre la matriz de entrada y los pesos.
- **Pista:** Revisa la documentación de `np.dot`.

Paso 2: La Función de Activación (TODO 3)

Busca el método `predict(self, X)`.

El Perceptrón simple utiliza una función de escalón (Heaviside step function) para decidir si se "dispara" o no.

- **Tu tarea:** Implementa la lógica para devolver:

- 1 si la entrada neta (que calculaste en el paso anterior) es mayor o igual a 0.
- 0 si es menor que 0.
- **Pista:** Puedes usar un if/else clásico, pero como expertos en Big Data, se recomienda usar `np.where(condición, valor_si_true, valor_si_false)` para que tu código sea vectorizable en el futuro.

Paso 3: El Aprendizaje (TODO 1)

Busca el método `fit(self, X, y)`.

Aquí ocurre la magia. La neurona iterará sobre los datos de entrenamiento varias veces (épocas) para ajustar sus pesos.

Dentro del bucle, para cada muestra, debes implementar estos 3 sub-pasos:

1. **Predicción:** Usa el método `predict` que acabas de crear para ver qué "piensa" la neurona con sus pesos actuales.
2. **Cálculo del Error (Update):** Compara la predicción con la realidad.
 - La fórmula de actualización es: $\Delta w = \eta \cdot (y_{real} - y_{predicho})$
 - Donde η es `self.learning_rate`.
3. **Actualización de Pesos y Sesgo:**
 - Ajusta los pesos actuales sumándoles el error ponderado por la entrada:

$$w \leftarrow w + \Delta w \cdot x$$
 - Ajusta el sesgo sumándole el error: $b \leftarrow b + \Delta w$

Validación y Pruebas

Al final del fichero encontrarás un bloque `if __name__ == "__main__":` preparado con la **Tabla de verdad de la compuerta AND**.

1. Ejecuta tu script.
2. Si has implementado todo correctamente:
 - No debería haber errores de ejecución.
 - El programa imprimirá los **Pesos finales** y el **Bias final**.
 - Verás una prueba final donde la entrada `[1, 1]` predice 1.
3. Si obtienes errores o la predicción falla, revisa tus fórmulas.